Lab Assignment 5: Filter Bank Design

K Shiva Gnana Yeseswini —BL.EN.U4AIE23013

19-03-2026

1 Objective

- To understand how to construct a triangular filter
- To learn how multiple triangular filters form a filter bank
- To design and visualize a Mel Filter Bank
- To understand the need for FFT bin mapping

2 Theory

A triangular filter is defined using three points:

- Left point (a) – start of filter
- Center point (b) – peak of filter
- Right point (c) – end of filter

The filter increases linearly from a to b and decreases linearly from b to c, and is zero outside this range.

$$H(k) = \begin{cases} 0 & k < a \\ \frac{k-a}{b-a} & a \leq k \leq b \\ \frac{c-k}{c-b} & b \leq k \leq c \\ 0 & k > c \end{cases}$$

The Mel scale represents human perception of sound:

$$Mel(f) = 2595 \log_{10}\left(1 + \frac{f}{700}\right)$$

$$f = 700\left(10^{\frac{Mel}{2595}} - 1\right)$$

FFT bins are related to frequency by:

$$f_k = \frac{k f_s}{N}$$

3 Implementation

3.1 Triangular Filter Construction

The triangular filter was implemented using a piecewise function. Both single and multiple filters were plotted to observe their behavior.

Code Implementation:

```
    for i in range(len(k)):
        if k[i] < a:
            H[i] = 0
        elif a <= k[i] <= b:
            H[i] = (k[i] - a) / (b - a)
        elif b <= k[i] <= c:
            H[i] = (c - k[i]) / (c - b)
        else:
            H[i] = 0
    return H

# k-axis
k = np.linspace(0, 60, 500)

# (i) Single triangle
a, b, c = 10, 20, 30
H = triangular_filter(k, a, b, c)

plt.plot(k, H)
plt.title("Single Triangular Filter")
plt.xlabel("k")
plt.ylabel("H(k)")
plt.grid()
plt.show()

# (ii) Multiple triangles
triangles = [(10,20,30), (20,30,40), (30,40,50)]
```

Figure 1: Triangular Filter Code

Single Triangular Filter Output:

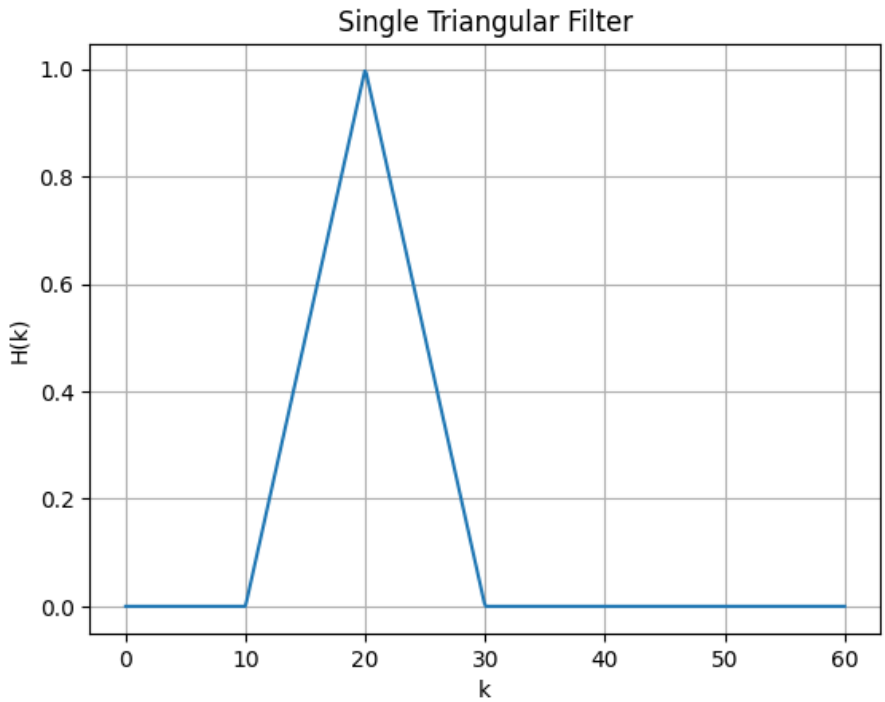

Figure 2: Single Triangular Filter

Multiple Triangular Filters Output:

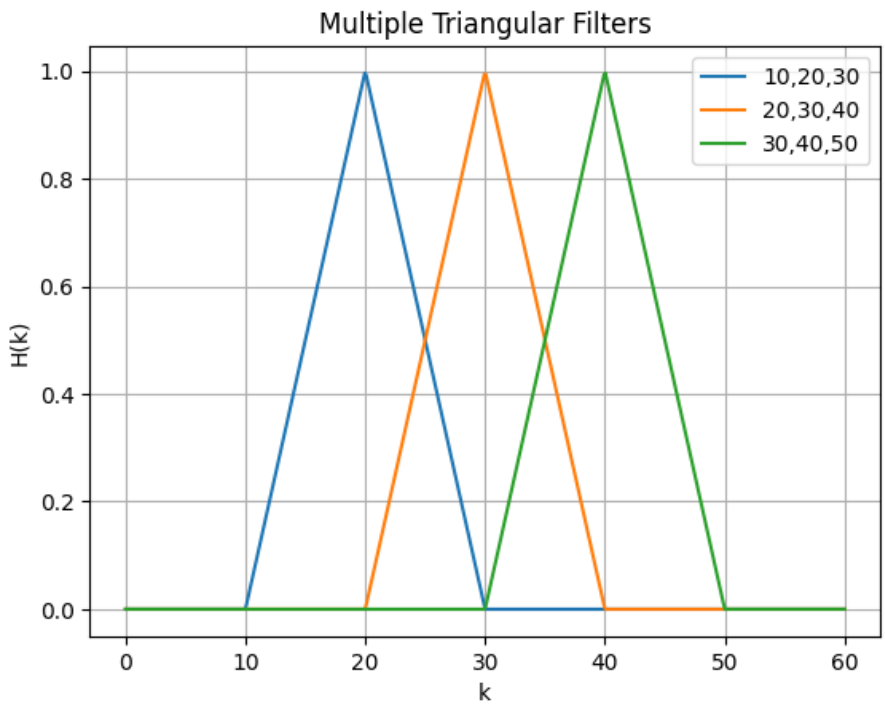

Figure 3: Multiple Triangular Filters

Explanation: From a to b, the filter increases linearly, forming the rising edge. From b to c, it decreases linearly, forming the falling edge. Multiple filters overlap to ensure smooth frequency coverage.

3.2 Mel Filter Bank Design

The Mel filter bank was constructed by converting frequency limits to the Mel scale, generating equally spaced Mel points, converting them back to frequency, and mapping them to FFT bins.

Code Implementation:

```
f_min = 0
f_max = fs / 2

mel_min = hz_to_mel(f_min)
mel_max = hz_to_mel(f_max)

mel_points = np.linspace(mel_min, mel_max, M + 2)
```

```
hz_points = mel_to_hz(mel_points)
```

```
bins = np.floor((N + 1) * hz_points / fs).astype(int)
```

```
filter_bank = np.zeros((M, int(N/2 + 1)))

for m in range(1, M + 1):
    f_m_minus = bins[m - 1]
    f_m = bins[m]
    f_m_plus = bins[m + 1]

    for k in range(f_m_minus, f_m):
        filter_bank[m-1, k] = (k - f_m_minus) / (f_m - f_m_minus)

    for k in range(f_m, f_m_plus):
        filter_bank[m-1, k] = (f_m_plus - k) / (f_m_plus - f_m)
```

Figure 4: Mel Filter Bank Code

Mel Filter Bank Output:

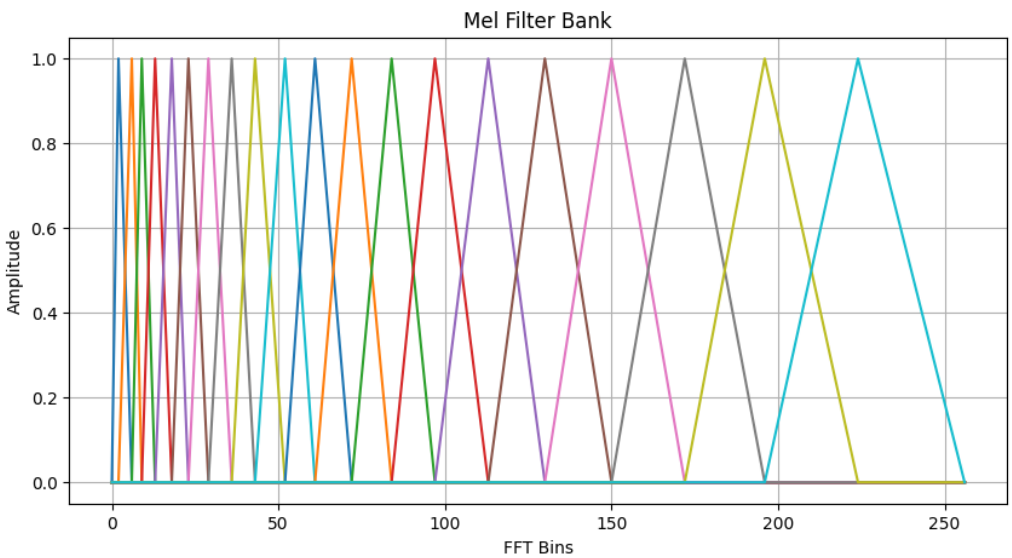

Figure 5: Mel Filter Bank

Explanation: The filters are equally spaced in the Mel scale but appear non-uniform in the frequency domain. Each filter overlaps with adjacent filters, ensuring smooth representation of frequency components.

4 Results and Analysis

4.1 Mel Points Visualization

```
plt.plot(mel_points, 'o-')
plt.title("Mel Points")
plt.xlabel("Index")
plt.ylabel("Mel Value")
plt.grid()
plt.show()
```

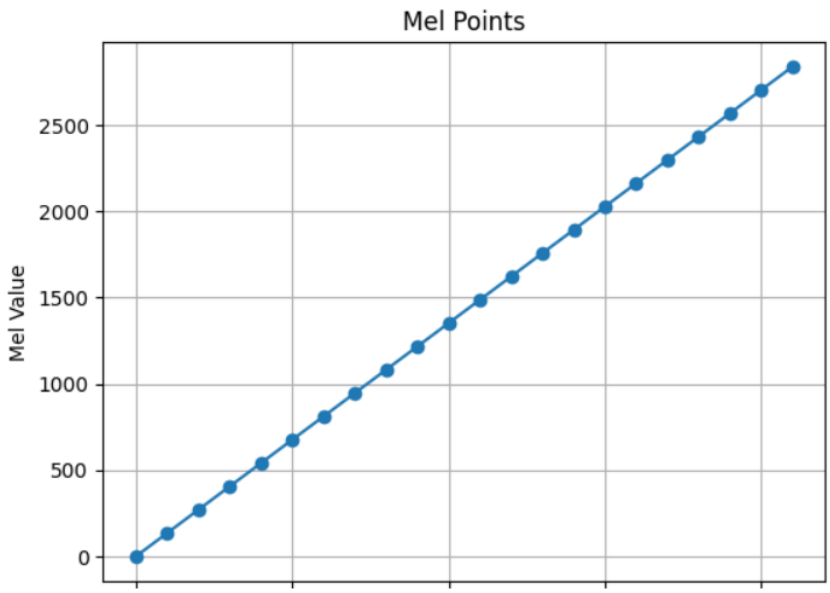

Figure 6: Mel Points

Observation: Mel points are evenly spaced, indicating uniform perception in the Mel scale.

4.2 Triangular Filter Parameters

```python
for i in range(1, M+1):
    print(f"Filter {i}: a={bins[i-1]}, b={bins[i]}, c={bins[i+1]}")
```

```
Filter 1: a=0, b=2, c=6
Filter 2: a=2, b=6, c=9
Filter 3: a=6, b=9, c=13
Filter 4: a=9, b=13, c=18
Filter 5: a=13, b=18, c=23
Filter 6: a=18, b=23, c=29
Filter 7: a=23, b=29, c=36
Filter 8: a=29, b=36, c=43
Filter 9: a=36, b=43, c=52
Filter 10: a=43, b=52, c=61
Filter 11: a=52, b=61, c=72
Filter 12: a=61, b=72, c=84
Filter 13: a=72, b=84, c=97
Filter 14: a=84, b=97, c=113
Filter 15: a=97, b=113, c=130
Filter 16: a=113, b=130, c=150
Filter 17: a=130, b=150, c=172
Filter 18: a=150, b=172, c=196
Filter 19: a=172, b=196, c=224
Filter 20: a=196, b=224, c=256
```

Figure 7: Filter Parameters (a, b, c)

Observation: The values of (a, b, c) increase progressively and are not uniformly spaced in Hz, confirming the non-linear nature of the Mel scale.

5 Key Observations

- Triangular filters exhibit linear rise and fall characteristics.
- Overlapping filters ensure no loss of frequency information.
- Mel scale reflects human auditory perception more accurately than linear frequency.
- Filter spacing is uniform in Mel scale but non-uniform in Hz.
- FFT bin mapping is essential because digital signals are processed in discrete form.

6 Conclusion

In this experiment, triangular filters were successfully constructed and extended to form a filter bank. A Mel filter bank was implemented using frequency-to-Mel conversion and FFT bin mapping.

This experiment demonstrated how human auditory perception can be modeled using signal processing techniques and highlighted the importance of discrete frequency representation in practical implementations.

7 Learning Outcomes

- Understood the mathematical formulation of triangular filters
- Learned how filter banks are constructed using overlapping filters
- Gained insight into the Mel scale and its importance in audio processing
- Understood the role of FFT bins in digital signal processing
- Developed practical skills in implementing filter banks using Python